

15/06

### MODULE 3

## DIVIDE & CONQUER and GREEDY STRATEGY

*xxxxxxxxxxxxxxxxxxxxxxxxxxxx*

## Divide and Conquer

\* DIVIDE

Divide the problem into smaller sub problems, that were smaller instances of the original problem.

\* CONQUER

Solve the sub problems recursively

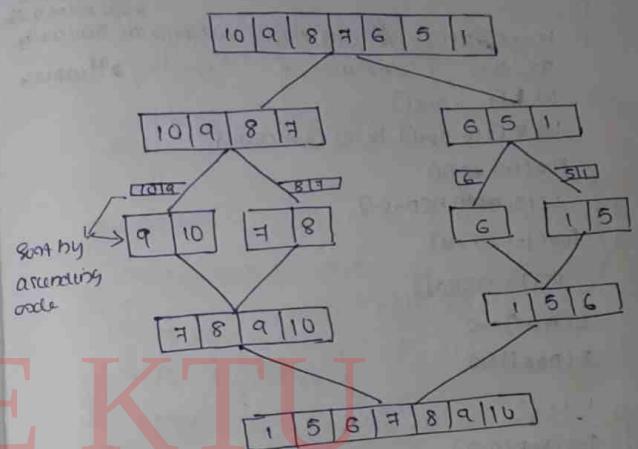
\* COMPINTE

combine the solved subproblems to get the final solution

### \* CONTROL ABSTRACTION

By a control abstraction we need a procedure whose flow of control is clear, but whose primary operation is specified by other procedure, whose precise meaning is left undefining.

Example of divide & conquer strategy is Merge Sort



# 16/06 Merge Sort - algorithm

## (1) merging

MERGE(A, p, q, r)

{

$n_1 = q - p + 1$

//  $n_1$  is no. of elements in first array

$n_2 = r - q$

//  $n_2$  is no. of elements in second array

let L[1...n<sub>1</sub>]

let R[1 to n<sub>2</sub>+1] be a new array

for (i=1 to n<sub>1</sub>)

L[i] = A[p+i-1]

for (j=1 to n<sub>2</sub>)

R[j] = A[q+j]

L[n<sub>1</sub>+1] = ∞

R[n<sub>2</sub>+1] = ∞

i=1, j=1

for (k=p to r)

if L[i] ≤ R[j]

A[k] = L[i]

i = i + 1

k++

else

A[k] = R[j]

j = j + 1

k++

}

A - array

p -> first array q -> q, is the last position of p array

A -> array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

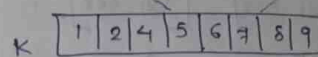
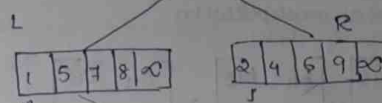
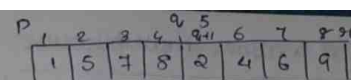
q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array

q+1 on the second array's first position is the first position of second array



## MERGE SORT

(2)

MERGE-SORT(A, p, r)

{

if p < r

q = ⌊(p+r)/2⌋

MERGE-SORT(A, p, q)

MERGE-SORT(A, q+1, r)

MERGE(A, p, q)

}

Time complexity =  $T(n) = 2T(n/2) + O(n)$

using master theorem to compute complexity

a=2 b=2 k=0  $T(n) = O(n \log n)$

$a > b^k$   $T(n) = O(n^{\log_b a}) = O(n^{\log_2 2}) = O(n)$

$a = b^k$   $T(n) = O(n^{\log_b a} \log n) = O(n \log n)$

$a < b^k$   $T(n) = O(n^{\log_b a}) = O(n^{\log_2 2}) = O(n)$

$a < b^k$   $T(n) = O(n^{\log_b a}) = O(n^{\log_2 2}) = O(n)$

## Strassen's matrix multiplication

In normal multiplication

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

A        ×        B        =        C

$$c_{11} = (a_{11} * b_{11}) + (a_{12} * b_{21})$$

$$c_{12} = (a_{11} * b_{12}) + (a_{12} * b_{22})$$

$$c_{21} = (a_{21} * b_{11}) + (a_{22} * b_{21})$$

$$c_{22} = (a_{21} * b_{12}) + (a_{22} * b_{22})$$

Time complexity analysis

In normal matrix multiplication takes 8 different multiplication tasks & stages along with 4 additions

We know that the time complexity for multiplication is greater than that of addition

time complexity required for addition is  $O(n^2)$

$$\therefore T(n) = 8T(n/2) + O(n^2)$$

using master's theorem

$$T(n) = O(n^3)$$

$$\begin{matrix} a=8 & p=0 \\ b=2 & a > b^k, 8 > 2^3 \\ k=2 \end{matrix}$$

$$a > b^k \Rightarrow O(n^{\log_b a})$$

$$T(n) = O(n^{\log_2 8})$$

$$= O(n^3)$$

\* Strassen introduced a new matrix multiplication

strategy based on divide and conquer method

\* Here, Strassen reduces the number of multiplications to seven

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

then

$$D = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$Q = b_{11}(a_{21} + a_{22})$$

$$R = a_{11}(b_{12} - b_{22})$$

$$S = a_{22}(b_{21} - b_{11})$$

$$T = b_{22}(a_{11} + a_{12})$$

$$U = (b_{11} + b_{12})(a_{21} - a_{11})$$

$$V = (b_{21} + b_{22})(a_{12} - a_{22})$$

$$c_{11} = D + S - T + V$$

$$c_{12} = R + T$$

$$c_{21} = Q + S$$

$$c_{22} = D + R - Q + U$$

Analysis of time complexity

$$T(n) = 7T(n/2) + O(n^2) \text{ using master's Theorem}$$

$$T(n) = n^{\log_2 7}$$

$$Q: \begin{bmatrix} 1 & 9 \\ 7 & 5 \end{bmatrix} \times \begin{bmatrix} 6 & 7 \\ 3 & 8 \end{bmatrix} = ?$$

$$P = (a_{11} + a_{22})(b_{11} + b_{22}) \quad \text{Ans: } \begin{bmatrix} 15 & 31 \\ 57 & 89 \end{bmatrix}$$

$$= (1 + 5)(6 + 8)$$

$$= (6)(14)$$

$$= \underline{84}$$

$$Q = b_{11}(a_{21} + a_{22}) \quad U = (b_{11} + b_{12})(a_{21} - a_{11})$$

$$= 6 \times (7 + 5)$$

$$= 6 \times 12$$

$$= \underline{72}$$

$$= (6 + 7)(7 - 1)$$

$$= (13 \times 6) = \underline{78}$$

$$R = a_{11}(b_{12} - b_{22}) \quad V = (b_{21} + b_{22})(a_{12} - a_{22})$$

$$= 1 \times (7 - 8)$$

$$= -1$$

$$= (3 + 8)(3 - 5)$$

$$= 11 \times -2$$

$$= \underline{-22}$$

$$S = a_{22}(b_{21} - b_{11}) \quad C_{11} = P + S - T + V$$

$$= 5 \times (3 - 6)$$

$$= 5 \times -3$$

$$= \underline{-15}$$

$$= 84 + -15 - 30 - 22$$

$$= \underline{15}$$

$$T = b_{22}(a_{11} + a_{12}) \quad C_{12} = R + T$$

$$= 8 \times (1 + 9)$$

$$= 8 \times 10$$

$$= \underline{80}$$

$$= -1 + 82 = \underline{81}$$

$$C_{21} = Q + S$$

$$= 72 + -15 = \underline{57}$$

$$C_{22} = P + R - Q + U$$

$$= 84 + -1 - 72 + 78 = \underline{89}$$

$$Q: \begin{bmatrix} 3 & 2 \\ 7 & 5 \end{bmatrix} \times \begin{bmatrix} 6 & 7 \\ 3 & 8 \end{bmatrix} = ?$$

$$P = (a_{11} + a_{22})(b_{11} + b_{22}) \quad V = (b_{21} + b_{22})(a_{12} - a_{22})$$

$$= (3 + 5)(6 + 8)$$

$$= 8 \times 14$$

$$= \underline{112}$$

$$= (3 + 8)(2 - 5)$$

$$= 11 \times -3 = \underline{-33}$$

$$Q = b_{11}(a_{21} + a_{22}) \quad C_{11} = P + S - T + V$$

$$= 6 \times (7 + 5)$$

$$= 6 \times 12$$

$$= \underline{72}$$

$$= 112 + (-15) - 40 + (-33)$$

$$= \underline{24}$$

$$R = a_{11}(b_{12} - b_{22}) \quad C_{12} = R + T$$

$$= 3 \times (7 - 8)$$

$$= 3 \times -1$$

$$= \underline{-3}$$

$$= -3 + 40$$

$$= \underline{37}$$

$$S = a_{22}(b_{21} - b_{11}) \quad C_{21} = Q + S$$

$$= 5 \times (3 - 6)$$

$$= 5 \times -3 = \underline{-15}$$

$$= 72 + -15 = \underline{57}$$

$$T = b_{22}(a_{11} + a_{12}) \quad C_{22} = P + R - Q + U$$

$$= 8 \times (3 + 2)$$

$$= 8 \times 5 = \underline{40}$$

$$= 112 + (-6) - 72 + 52$$

$$= \underline{89}$$

$$U = (b_{11} + b_{12})(a_{21} - a_{11}) \quad C = \begin{bmatrix} 24 & 37 \\ 57 & 89 \end{bmatrix}$$

$$= (6 + 7)(7 - 3)$$

$$= 13 \times 4 = \underline{52}$$

18/06

Greedy Strategy (Technique)

It is an algorithmic paradigm that follows the problem solving approach of making the locally optimal choice at each stage with the hope of finding a global optimum.

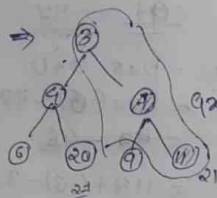
Pros / Advantages

Simple, easy to code and implement

It runs fast

Cons / Disad

Very often they don't provide a global optimum solution



Greedy method

constraint: path with maximum node sum

\* making locally optimal choice doesn't give an global optimal solution.  $\therefore$

\* Greedy approach works based on two properties

(i) Greedy choice

A global optimum can be achieved by selecting a local global optimum.

(ii) Optimal Substructure

An optimal solution to the problem contains an optimal solution to the subproblem.

Applications using Greedy Strategy

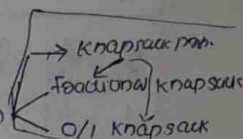
\* Huffman coding

\* fractional knapsack problem.

\* job sequencing problem.

\* Activity selection problem.

\* Fractional knapsack problem



Q:

| Object        | 1  | 2   | 3  | 4 | 5 | 6   | 7 |
|---------------|----|-----|----|---|---|-----|---|
| Profit        | 10 | 5   | 15 | 7 | 6 | 18  | 3 |
| Weight        | 2  | 3   | 5  | 7 | 1 | 4   | 1 |
| Profit/Weight | 5  | 1.6 | 3  | 1 | 6 | 4.5 | 3 |

$\frac{1.6 \times 3}{4.8}$

Weight of knapsack = 15 aim: to get max. profit

| Knapsack weight | items in knapsack                  | cost          |
|-----------------|------------------------------------|---------------|
| 15              | $\emptyset$                        | $\emptyset$   |
| $15-1=14$       | $O_5$                              | 6             |
| $14-2=12$       | $\{O_5, O_1\}$                     | $6+10=16$     |
| $12-4=8$        | $\{O_5, O_1, O_6\}$                | $16+18=34$    |
| $8-5=3$         | $\{O_5, O_1, O_6, O_3\}$           | $34+15=49$    |
| $3-1=2$         | $\{O_5, O_1, O_6, O_3, O_1\}$      | $49+5=54$     |
| 0               | $\{O_5, O_1, O_6, O_3, O_1, O_2\}$ | $54+5+52=111$ |

③ ③ ③ ① ③ ③  
 $O_1$   $O_2$   $O_3$   $O_4$   $O_5$   $O_6$   $O_7$   
 1 2/3 1 0 1 1 1

9. Find the optimal solution for the fractional knapsack problem using Greedy approach. Capacity of the knapsack is 60kg. The seller contains 5 distinct items. Their weights are 5, 10, 15, 20, 25 values are 30, 40, 45, 77, 90

| Object       | 1  | 2  | 3  | 4    | 5   |
|--------------|----|----|----|------|-----|
| Value        | 30 | 40 | 45 | 77   | 90  |
| Weight       | 5  | 10 | 15 | 20   | 25  |
| Value/Weight | 6  | 4  | 3  | 3.85 | 3.6 |

| Knapsack weight | items in knapsack        | cost         |
|-----------------|--------------------------|--------------|
| 60 kg           | $\emptyset$              | $\emptyset$  |
| $60-5=55$       | $O_1$                    | 30           |
| $55-10=45$      | $\{O_1, O_2\}$           | $30+40=70$   |
| $45-15=30$      | $\{O_1, O_2, O_3\}$      | $70+45=115$  |
| $30-20=10$      | $\{O_1, O_2, O_3, O_4\}$ | $115+77=192$ |
| $10-25=-15$     |                          |              |

0/1 knapsack

| item          | 1  | 2   | 3  | 4 | 5 | 6   | 7 |
|---------------|----|-----|----|---|---|-----|---|
| profit        | 10 | 5   | 15 | 7 | 6 | 18  | 3 |
| weight        | 2  | 3   | 5  | 7 | 1 | 4   | 1 |
| profit/weight | 5  | 1.6 | 3  | 1 | 6 | 4.5 | 3 |

Total knapsack capacity = 15  
 arrange the items in knapsack (0/1) to get maximum profit.

Q18:

| Knapsack capacity | item in knapsack          | profit      |
|-------------------|---------------------------|-------------|
| 15                | $\emptyset$               | $\emptyset$ |
| $15-1=14$         | $I_5$                     | 6           |
| $14-2=12$         | $I_5, I_1$                | $6+10=16$   |
| $12-9=3$          | $I_5, I_1, I_6$           | $16+18=34$  |
| $3-5=2$           | $I_5, I_1, I_6, I_3$      | $34+15=49$  |
| $2-1=1$           | $I_5, I_1, I_6, I_3, I_4$ | $49+3=52$   |

$I_1, I_2, I_3, I_4, I_5, I_6, I_7$  profit = 52  
 1 1 1 1 1 1 1 knapsack = 2

### ⇒ Analysis of knapsack problem

- 1) For  $i=1$  to  $n$ , profit calculation  $\rightarrow O(n)$   
 (weight)
  - 2) Sorting of  $i$  item for  $i=1$  to  $n$   
 using quick sort  $O(n \log n)$
  - 3) Adding the items and reducing the weight of knapsack  $\rightarrow O(n)$
- Total time complexity =  $O(n) + O(n \log n) + O(n)$   
 $T(n) = O(n \log n)$

Q19:

### Spanning tree

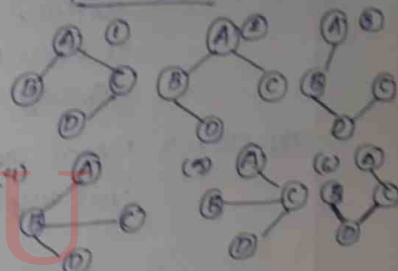
Consider  $G$  is a graph where  $G=(V, E)$   
 in which  $G'$  is a spanning tree generated from  $G$  such that  $G'=(V, E')$ , where  
 $V'=V$  and  $E' \subseteq E$  ( $E'$  is the subset of  $E$ ),  
 $E' = |V|-1$

eg: Graph  $G$

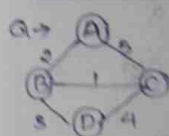


here  $V=4$   
 $E=5$

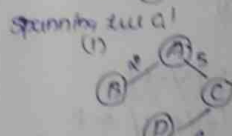
### Spanning tree $G'$



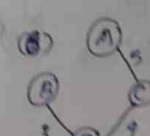
### MINIMUM SPANNING TREE (MST)



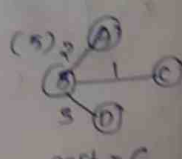
minimum spanning tree contains minimum total cost or edges are minimum.



cost = 11



cost = 12



cost = 6

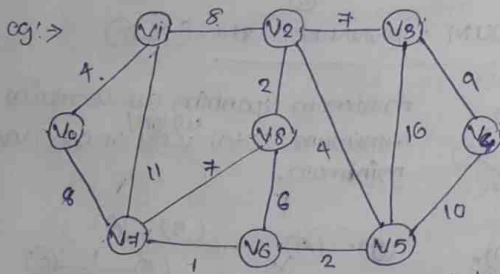
↑ this is the minimum spanning tree

# Kruskal's algorithm for finding MST

Step (1) → Sort all the edges in <sup>(Increasing)</sup> non-decreasing order of their weights.

Step (2) → pick the smallest edge. check if it forms a cycle with the spanning tree formed so far, If cycle is not form include this edge else discarding it.

Step (3) → Repeat step (2) until there are  $(n-1)$  edges in the spanning tree



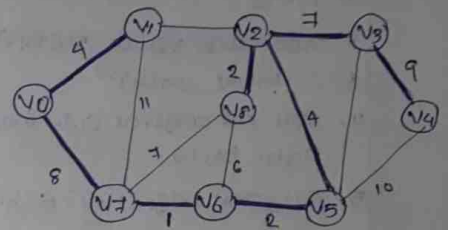
$$E = (V-1)$$

$$E' = |9-1| = 8$$

ans: Step (1):

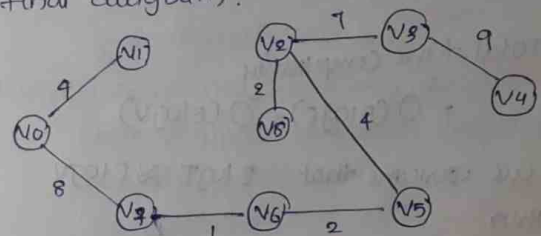
|       |    |           |
|-------|----|-----------|
| V7 V6 | 1  | ✓         |
| V2 V8 | 2  | ✓         |
| V6 V5 | 2  | ✓         |
| V0 V1 | 4  | ✓         |
| V2 V5 | 4  | ✓         |
| V6 V8 | 6  | ✓         |
| V7 V8 | 7  | ✓         |
| V2 V3 | 7  | ✓         |
| V0 V3 | 8  | ✓         |
| V0 V2 | 8  | discarded |
| V3 V4 | 9  | ✓         |
| V4 V5 | 10 | discarded |
| V1 V7 | 11 | discarded |
| V3 V5 | 16 | discarded |

Step (2):



Cost = 37

Final diagram:



Total cost = 37

### Pseudo code

MST\_KRUSKAL( $G, W$ )

1.  $A = \emptyset \rightarrow O(1)$
2. For each vertex  $v \in G.V$
3. MAKE-SET( $v$ ) } slow function
4. Sort the edges of  $G.E$  into non-decreasing order by  $w$ . }  $O(E \log E)$
5. For each edge  $(u, v) \in G.E$  taken in non-decreasing order by weight }  $O(E \log V)$
6. IF FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7.      $A \cup \{(u, v)\}$
8.     UNION( $u, v$ )
9. Return  $A$

Total time complexity

$$= O(E \log E) + O(E \log V)$$

We assume that  $E \log E \approx E \log V$

then

Time complexity of Kruskal's algorithm is,

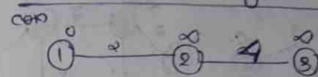
either  $E \log E$  or  $E \log V$

22/10/16

### Single Source Shortest Path Algorithm

Given a graph, which is connected and weighted  $G = (V, E)$ , Find the shortest path from a given source vertex  $s \in V$ , to every other vertex  $v \in V$  in the graph.

### Dijkstra's Algorithm



Consider the graph, in which source vertex is one. To find the shortest path to all other nodes from the source node follow the steps.

Step (1): mark the distance of source vertex as 0, also mark the distance of all other vertices to  $\infty$

Step (2): From the source node, find out the minimum path towards a vertex and follow the relaxation procedure

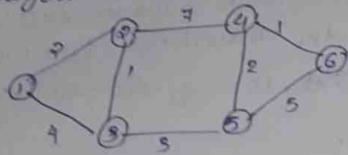
Relaxation

$$if (d[u] + c(u, v) < d[v])$$

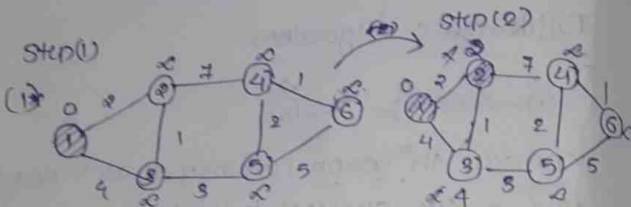
$$d[v] = d[u] + c(u, v)$$

Step (3): continue the step (2) until all the vertices get selected.

Q:1 Find the shortest path using Dijkstra's algorithm from the source vertex (1)

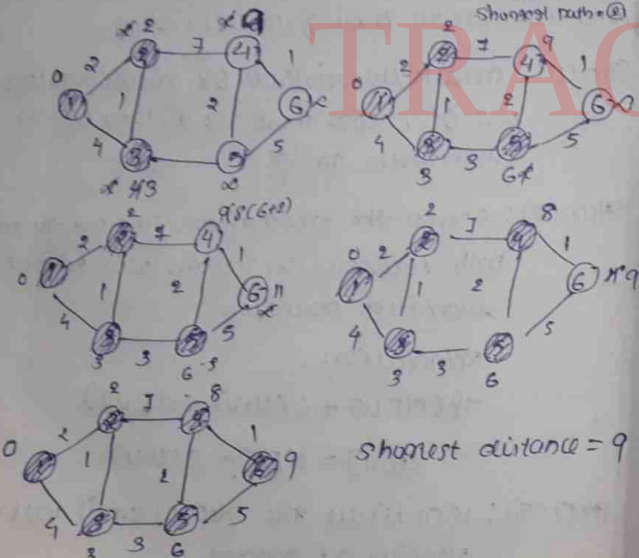


Step (1)



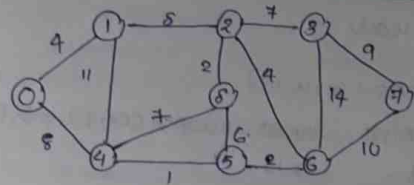
Step (2)

Sub (normal) in  
Shortest path = 6

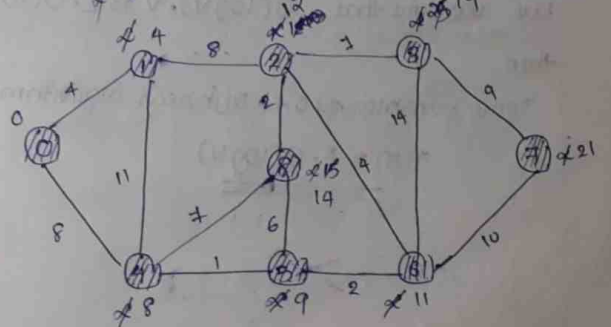
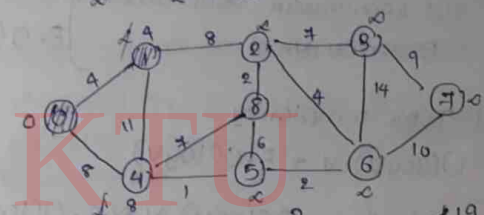
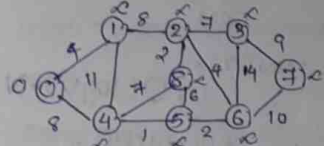


Shortest distance = 9

Q:2



Find the shortest path b/w node 0 to 7



Shortest path = 21

### Pseudo code

Dijkstra's (G, w, s)

1. INITIALISE\_SINGLE\_SOURCE (G, s)  $\rightarrow O(V)$
2.  $S = \emptyset \rightarrow O(1)$
3.  $Q = G.V$  — build a heap —  $O(V)$
4. WHILE  $Q \neq \emptyset \rightarrow O(1)$
5.  $U = \text{EXTRACT\_MIN}(Q)$
6.  $S = S \cup \{U\}$   $O(\log V) \cdot V$
7. FOR each vertex  $V \in G.V \setminus S$
8. RELAX (U, V, w)  $E \cdot O(\log V)$

Total time complexity  
 $= O(\log V) \cdot V + E \cdot O(\log V)$

We assume that,  $O(\log V) \cdot V \approx E \cdot O(\log V)$

then

Time complexity of Dijkstra's algorithm is,

$$\underline{\underline{T(n) = E \cdot O(\log V)}}$$

28106

### MODULE 4

#### DYNAMIC PROGRAMMING

$\rightarrow$  Divide & conquer strategy

$\rightarrow$  It is used to find out the optimal solution of a problem dynamic programming is applied when the subproblem overlaps.

$\rightarrow$  Steps

- (1) Characteristic the structure of an optimal solution
- (2) Recursively defined the value of an optimal sol?
- (3) Compute the value of an optimal sol?
- (4) Construct optimal solution from computed information.

int fib(n)

eg: fibonacci series

{ if (n < 1)

return n;

else

return (fib(n-2) + fib(n-1))

}

## Matrix chain multiplication

\* Given a sequence of matrices, find the most efficient way to multiply these matrices together.

\* There is no need to actually perform the multiplication.

\* We only need to decide the order of multiplication.

eg! How else have 4 different matrices  $A_1, A_2, A_3, A_4$ .  
We have to find the efficient order to multiply these matrices.

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4$$

$$\rightarrow (A_1 \cdot A_2) \cdot (A_3 \cdot A_4)$$

$$\rightarrow A_1 \cdot [(A_2 \cdot A_3) \cdot A_4]$$

$$\rightarrow [(A_1 \cdot A_2) \cdot A_3] \cdot A_4$$

$$\begin{matrix} 2 \times 3 & & 3 \times 4 & & 2 \times 4 \\ \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \end{bmatrix} & \begin{bmatrix} 1 & 2 & 3 & 0 \\ 3 & 2 & 1 & 0 \\ 1 & 3 & 4 & 1 \end{bmatrix} & = & \begin{bmatrix} 10 & 15 & 17 & 8 \\ 12 & 13 & 13 & 1 \end{bmatrix} \end{matrix}$$

$$= \begin{bmatrix} 1+6+9 & 2+4+9 & 3+2+12 & 0+0+8 \\ 2+9+3 & 4+6+9 & 6+3+4 & 0+0+1 \end{bmatrix}$$

Total 24 multiplication.

Let

| Matrix   | Dimension            |
|----------|----------------------|
| $M_1$    | $P_0 \times P_1$     |
| $M_2$    | $P_1 \times P_2$     |
| $M_3$    | $P_2 \times P_3$     |
| $\vdots$ | $\vdots$             |
| $M_n$    | $P_{n-1} \times P_n$ |

$$M(P_0, P_1, P_2, \dots, P_{n-1}, P_n)$$

Find

- 1) minimum no. of operations
- 2) minimum cost / minimum computation time
- 3) Sequence / Optimal parenthesis

$$M[i, j] = \min_{i \leq k < j} \begin{cases} 0 & \text{if } i=j \\ m[i, k] + m[k+1, j] + P_{i-1} \cdot P_k \cdot P_j \end{cases}$$

Q! Find the matrix multiplication order.

$$\left. \begin{array}{l} A \rightarrow 20 \times 2 \\ B \rightarrow 2 \times 15 \\ C \rightarrow 15 \times 40 \\ D \rightarrow 40 \times 4 \end{array} \right\} \text{OR } \boxed{20 \mid 2 \mid 15 \mid 40 \mid 4}$$

am!

$$P_0 \Rightarrow 20$$

$$P_1 \Rightarrow 9$$

$$P_2 \Rightarrow 15$$

$$P_3 \Rightarrow 40$$

$$P_4 \Rightarrow 4$$

using tabular form

| i \ j | 1    | 2    | 3    | 4 |
|-------|------|------|------|---|
| 1     | 1680 | 1520 | 2400 | 0 |
| 2     | 2800 | 1200 | 0    |   |
| 3     | 600  | 0    |      |   |
| 4     | 0    |      |      |   |

sequence! based on the value of k

| i \ j | 1 | 2 | 3 |
|-------|---|---|---|
| 1     | 1 | 3 | 3 |
| 2     | 1 | 2 |   |
| 3     | 1 |   |   |

$$M[i, j] = \min_{1 \leq k < j} \{ 0 \text{ if } i=j, M[i, k] + M[k+1, j] + P_{i-1} \cdot P_k \cdot P_j \}$$

$$M[1, 1] = i=j = 0$$

$$M[2, 2] = i=j = 0$$

$$M[3, 3] = i=j = 0$$

$$M[4, 4] = i=j = 0$$

$$M[1, 2] = \min_{1 \leq k < 2} \{ M[1, 1] + M[2, 2] + P_0 \cdot P_1 \cdot P_2 \}$$

$$= \min_{1 \leq k < 2} \{ 0 + 0 + P_0 \cdot P_1 \cdot P_2 \}$$

$$= \min_{1 \leq k < 2} \{ 20 \times 9 \times 15 = 600 \}$$

$$M[2, 3] = \min_{2 \leq k < 3} \{ M[2, 2] + M[3, 3] + P_1 \cdot P_k \cdot P_j \}$$

$$= \{ 0 + 0 + P_1 \cdot P_2 \cdot P_3 \}$$

$$= \{ P_1 \cdot P_2 \cdot P_3 \}$$

$$= \{ 9 \times 15 \times 40 = 1200 \}$$

$$M[1, 3] = \min_{1 \leq k < 3} \{ M[1, 2] + M[3, 3] + P_0 \cdot P_k \cdot P_3 \}$$

or

hence k=1

$$\{ 600 + 0 + P_0 \cdot P_2 \cdot P_3 \}$$

$$M[1, 1] + M[2, 3] = \{ 600 + 20 \cdot 15 \cdot 40 \}$$

$$+ P_0 \cdot P_1 \cdot P_3 = \{ 12000 + 20 \cdot 9 \cdot 40 = 12600 \}$$

$$= 0 + 1200 + 20 \times 2 \times 40 = 1200 + 1600 = 2800$$

$$M[3, 4] = \min_{3 \leq k < 4} \{ M[3, 3] + M[4, 4] + P_2 \cdot P_k \cdot P_4 \}$$

$$P_2 \cdot P_3 \cdot P_4$$

$$0 + 0 + P_2 \cdot P_3 \cdot P_4$$

$$15 \times 40 \times 4$$

$$= 2400$$

$$M[2, 4] = \min_{2 \leq k < 4} \{ M[2, 3] + M[4, 4] + P_1 \cdot P_k \cdot P_4 \}$$

k=3

$$1200 + 0 + 9 \times 40 \times 4$$

$$1200 + 1440$$

$$1440 = 1520$$

hence k=2

$$\min_{2 \leq k < 4} \{ M[2, 2] + M[3, 4] + P_1 \cdot P_k \cdot P_4 \}$$

$$0 + 2400 + 9 \times 15 \times 40$$

$$= 2400 + 1200 = 3600$$

$$m[1,4] = \min_{1 \leq k \leq 4} \{ m[1,3] + m[k,4] + P_0 \cdot P_3 \cdot P_4 \}$$

$$= \min_{k=3} \{ 2800 + 0 + 20 \times 40 \times 4 \}$$

$$= 2800 + 3200$$

$$= 6000$$

$$= \min_{1 \leq k \leq 4} \{ m[1,2] + m[k,4] + P_0 \cdot P_2 \cdot P_4 \}$$

$$= \min_{k=3} \{ 600 + 2400 + 20 \times 15 \times 4 \}$$

$$= 3000 + 1200$$

$$= 4200$$

$$= \min_{1 \leq k \leq 4} \{ m[1,1] + m[k,4] + P_0 \cdot P_1 \cdot P_4 \}$$

$$= \min_{k=3} \{ 0 + 1520 + 20 \times 2 \times 4 \}$$

$$= 1520 + 160$$

$$= 1680 \text{ min } 1680$$

1 2 3 4  
(A) ((B (C) D)

(A) ((B (C) D)

Q: Find the minimum way of multiplying matrices  $m_1, m_2, m_3, m_4, m_5$

|   |    |   |    |    |   |
|---|----|---|----|----|---|
| 4 | 10 | 3 | 12 | 20 | 7 |
|---|----|---|----|----|---|

$P_0 = 4$  A  $4 \times 10 \rightarrow M_1$   
 $P_1 = 10$  B  $10 \times 3 \rightarrow M_2$   
 $P_2 = 3$  C  $3 \times 12 \rightarrow M_3$   
 $P_3 = 12$  D  $12 \times 20 \rightarrow M_4$   
 $P_4 = 20$  E  $20 \times 7 \rightarrow M_5$   
 $P_5 = 7$

Tabular Form

| i \ j | 1    | 2    | 3    | 4    | 5 |
|-------|------|------|------|------|---|
| 5     | 1344 | 1350 | 1140 | 1680 | 0 |
| 4     | 1080 | 1320 | 720  | 0    |   |
| 3     | 264  | 360  | 0    |      |   |
| 2     | 120  | 0    |      |      |   |
| 1     | 0    |      |      |      |   |

Required: value of k

| i \ j | 1 | 2 | 3 | 4 |
|-------|---|---|---|---|
| 5     | 2 | 2 | 4 | 4 |
| 4     | 2 |   | 2 | 3 |
| 3     | 2 |   | 2 |   |
| 2     | 1 |   |   |   |

$$m[i,j] = \min_{1 \leq k < j} \{ 0 \text{ if } i=j; m[i,k] + m[k+1,j] + P_{i-1} \cdot P_k \cdot P_j \}$$

$$m[1,1] \Rightarrow i=j=0$$

$$m[2,2] \Rightarrow i=j=0$$

$$m[3,3] \Rightarrow i=j=0$$

$$m[4,4] \Rightarrow i=j=0$$

$$m[5,5] \Rightarrow i=j=0$$

$$m[1,2] = \min_{k=1} \{ m[1,1] + m[2,2] + P_0 \cdot P_1 \cdot P_2 \}$$

$$= 0 + 0 + 4 \times 10 \times 3 = \boxed{120}$$

$$m[2,3] = \min_{k=2} \{ m[2,2] + m[3,3] + P_1 \cdot P_2 \cdot P_3 \}$$

$$= 0 + 0 + 10 \times 3 \times 12 = \boxed{360}$$

$$m[3,4] = \min_{k=2} \{ m[3,2] + m[4,4] \}$$

$$m[3,4] = \min_{k=3} \{ m[3,3] + m[4,4] + P_2 \cdot P_3 \cdot P_4 \}$$

$$= 0 + 0 + 3 \times 12 \times 20 = \boxed{720}$$

$$m[4,5] = \min_{k=4} \{ m[4,4] + m[5,5] + P_3 \cdot P_4 \cdot P_5 \}$$

$$= 0 + 0 + 12 \times 20 \times 7 = \boxed{1680}$$

$$m[1,3] = \min_{k=2} \{ m[1,2] + m[3,3] + P_0 \cdot P_2 \cdot P_3 \}$$

$$= 120 + 0 + 4 \times 3 \times 12 = \boxed{264}$$

$$m[1,4] = \min_{k=3} \{ m[1,3] + m[4,4] + P_0 \cdot P_3 \cdot P_4 \}$$

$$= 0 + 360 + 4 \times 10 \times 12 = \boxed{840}$$

$$m[2,4] = \min_{k=3} \{ m[2,3] + m[4,4] + P_1 \cdot P_3 \cdot P_4 \}$$

$$= 360 + 0 + 10 \times 12 \times 20 = \boxed{2760}$$

$$m[2,5] = \min_{k=3} \{ m[2,4] + m[5,5] + P_1 \cdot P_4 \cdot P_5 \}$$

$$= 0 + 720 + 10 \times 3 \times 20 = \boxed{1320}$$

$$m[3,5] = \min_{k=4} \{ m[3,4] + m[5,5] + P_2 \cdot P_4 \cdot P_5 \}$$

$$= 720 + 0 + 3 \times 20 \times 7 = \boxed{1140}$$

$$m[3,5] = \min_{k=3} \{ m[3,3] + m[5,5] + P_2 \cdot P_3 \cdot P_4 \}$$

$$= 0 + 1680 + 3 \times 12 \times 7 = \boxed{1932}$$

$$m[1,4] = \min_{k=3} \{ m[1,3] + m[4,4] + P_0 \cdot P_3 \cdot P_4 \}$$

$$= 264 + 0 + 4 \times 12 \times 20 = \boxed{1224}$$

$$m[1,5] = \min_{k=2} \{ m[1,2] + m[5,5] + P_0 \cdot P_2 \cdot P_5 \}$$

$$= 120 + 720 + 4 \times 3 \times 20 = \boxed{1080}$$

$$m[2,5] = \min_{k=1} \{ m[2,1] + m[5,5] + P_1 \cdot P_1 \cdot P_5 \}$$

$$= 0 + 1320 + 4 \times 10 \times 20 = \boxed{2120}$$

$$m[2,5] = \min_{k=4} \{ m[2,4] + m[5,5] + P_1 \cdot P_4 \cdot P_5 \}$$

$$= 1920 + 0 + 10 \times 20 \times 7 = \boxed{2720}$$

$$m[3,5] = \min_{k=3} \{ m[3,3] + m[5,5] + P_2 \cdot P_3 \cdot P_5 \}$$

$$= 360 + 1680 + 10 \times 12 \times 7 = \boxed{2880}$$

$$m[3,5] = \min_{k=2} \{ m[3,2] + m[5,5] + P_2 \cdot P_2 \cdot P_5 \}$$

$$= 0 + 1140 + 10 \times 3 \times 7 = \boxed{1950}$$

$$m[1,5] = \min_{k=1} \{ m[1,1] + m[5,5] + P_0 \cdot P_1 \cdot P_5 \}$$

$$m[1,5] = \min_{k=4} \{ m[1,4] + m[5,5] + P_0 \cdot P_4 \cdot P_5 \}$$

$$= 1080 + 0 + 4 \times 20 \times 7 = \boxed{2480}$$

$$m[2,5] = \min_{k=3} \{ m[2,3] + m[5,5] + P_1 \cdot P_3 \cdot P_5 \}$$

$$= 264 + 1680 + 4 \times 12 \times 7 = \boxed{2820}$$

$$m[2,5] = \min_{k=2} \{ m[2,2] + m[5,5] + P_1 \cdot P_2 \cdot P_5 \}$$

$$= 120 + 1140 + 4 \times 3 \times 7 = \boxed{1944}$$

$$m[1,5] = \min_{k=1} \{ m[1,1] + m[5,5] + P_0 \cdot P_1 \cdot P_5 \}$$

$$= 0 + 1950 + 4 \times 10 \times 7 = \boxed{1690}$$

$$\begin{matrix} 1 & 2 & 3 & 4 & 5 \\ (A) & (B) & (C) & (D) & (E) \end{matrix} \quad \begin{matrix} 1 & 2 & 3 & 4 & 5 \\ (M_1) & (M_2) & (M_3) & (M_4) & (M_5) \end{matrix}$$

24/06

Time complexity Analysis of Matrix chain multiplication

→ For generating the table of 'n' elements

we take the complexity as  $\frac{n(n-1)}{2}$

→ Then we find minimum value among them

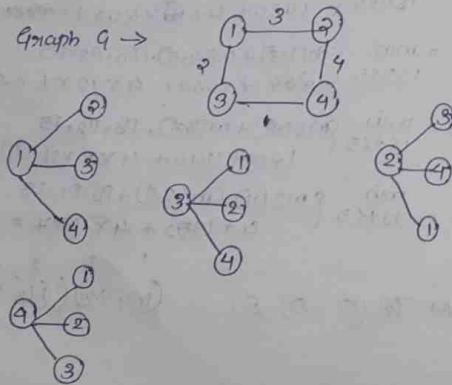
$$\frac{n(n-1)}{2} \times n = \frac{n^2 \cdot n}{2} = \frac{n^3}{2}$$

$$\therefore \text{Time} = O(n^3)$$

### All pair shortest path Algorithm

At a particular time for source node we have to find the shortest distance from that node to all other nodes.

Graph G →

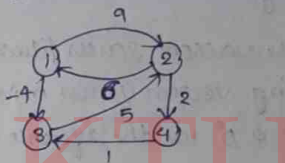


\* For the above eg. we do get the all pair shortest path we have to call Dijkstra's algorithm for 4 time.

\* To avoid the above problem we use the Floyd Warshall algorithm

\* Floyd Warshall algorithm is efficient in both +ve & -ve edge weight

### Floyd Warshall Algorithm



Step (1): Draw the distance matrix  $D_0$  by considering the direct edges

→ If we don't have direct edge put  $\infty$   
→ All diagonal elements should be 0.

$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 9 & -4 & 6 \\ \infty & 0 & \infty & 2 \\ \infty & 5 & 0 & \infty \\ \infty & \infty & 1 & 0 \end{bmatrix} \end{matrix}$$

Step (2): consider the vertex ① as middle element and find the next matrix  $D^1$  from  $D^0$

→ Here the selected vertex is ① so select the row and column of vertex ① from  $D^0$  distance matrix and write down to  $D^1$  matrix

→ Diagonal elements of  $D^1$  matrix should be 0

→ all other vacant spaces filled by considering vertex ① as middle element of  $D^0$  as the reference matrix

$$D^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 9 & -4 & \infty \\ 6 & 0 & 2 & 2 \\ \infty & 5 & 0 & \infty \\ \infty & \infty & 1 & 0 \end{bmatrix} \end{matrix}$$

$$D^0[2,3] = D^0[2,1] + D^0[1,3]$$

$$\text{middle element} = 6 + (-4) = 2 \quad \infty > 2 \text{ so } D^0[2,3] = 2$$

$$D^0[2,4] = D^0[2,1] + D^0[1,4]$$

$$2 = 6 + \infty = \infty \quad 2 < \infty \text{ so } 2$$

$$D^0[3,2] = D^0[3,1] + D^0[1,2]$$

$$5 = \infty + 9 = \infty$$

$$5 < \infty \text{ so } 5$$

$$D^0[4,2] = D^0[4,1] + D^0[1,2]$$

$$\infty = \infty + 9$$

$$\infty > \infty \text{ so } \infty$$

$$D^0[3,4] = D^0[3,1] + D^0[1,4]$$

$$\infty = \infty + \infty$$

$$\infty = \infty \text{ so } \infty$$

$$D^0[4,3] = D^0[4,1] + D^0[1,3]$$

$$1 = \infty + (-4)$$

$$1 < \infty \text{ so } 1$$

Step (3): consider the vertex ② as middle element and find the next matrix  $D^2$  from  $D^1$

$$D^2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 9 & -4 & \infty \\ 6 & 0 & 2 & 2 \\ 11 & 5 & 0 & 7 \\ \infty & \infty & 1 & 0 \end{bmatrix} \end{matrix}$$

middle element = 2

$$D^1[1,3] = D^1[1,2] + D^1[2,3]$$

$$\infty = 9 + 2$$

$$\infty > 11 \text{ so } 11$$

$$D^1[1,4] = D^1[1,2] + D^1[2,4]$$

$$\infty = 9 + 2$$

$$\infty > 11 \text{ so } 11$$

$$D^1[3,1] = D^1[3,2] + D^1[2,1]$$

$$\infty = 5 + 6$$

$$\infty > 11$$

$$D^1[3,4] = D^1[3,2] + D^1[2,4]$$

$$\infty = 5 + 2$$

$$\infty > 7$$

$$D^1[4,1] = D^1[4,2] + D^1[2,1]$$

$$\infty = \infty + 6$$

$$\infty > \infty$$

$$D^1[4,3] = D^1[4,2] + D^1[2,3]$$

$$1 = \infty + 2$$

$$1 < \infty$$

Step (4): Consider the vector (3) as middle element - & find the next matrix  $D^3$  from  $D^2$

|   | 1  | 2 | 3  | 4 |
|---|----|---|----|---|
| 1 | 0  | 1 | -4 | 5 |
| 2 | 6  | 0 | 2  | 2 |
| 3 | 11 | 5 | 0  | 7 |
| 4 | 12 | 6 | 1  | 0 |

$$D^3[1,2] = D^2[1,3] + D^2[3,2] \quad D^3[1,4] = D^2[1,3] + D^2[3,4]$$

$$9 = -4 + 5 \quad 11 = -4 + 7$$

$$9 > 1 \quad 11 > 3$$

$$D^3[2,1] = D^2[2,3] + D^2[3,1] \quad D^3[2,4] = D^2[2,3] + D^2[3,4]$$

$$6 = 2 + 11 \quad 2 = 2 + 7$$

$$6 < 13 \quad 2 < 9$$

$$D^3[4,1] = D^2[4,3] + D^2[3,1] \quad D^3[4,2] = D^2[4,3] + D^2[3,2]$$

$$8 = 1 + 11 \quad 8 = 1 + 5$$

$$8 > 12 \quad 8 > 6$$

Step (5): Consider the vector (4) as middle element & find the next matrix  $D^4$  from  $D^3$

|   | 1  | 2 | 3  | 4 |
|---|----|---|----|---|
| 1 | 0  | 1 | -4 | 5 |
| 2 | 6  | 0 | 2  | 2 |
| 3 | 11 | 5 | 0  | 7 |
| 4 | 12 | 6 | 1  | 0 |

$$D^3[1,2] = D^2[1,4] + D^2[4,2]$$

$$1 = -4 + 3 + 6$$

$$1 < 9$$

$$D^3[1,3] = D^2[1,4] + D^2[4,3]$$

$$-4 = 3 + 1$$

$$-4 < 4$$

$$D^3[2,1] = D^2[2,4] + D^2[4,1]$$

$$6 = 2 + 12$$

$$6 < 16$$

$$D^3[2,3] = D^2[2,4] + D^2[4,3]$$

$$2 = 2 + 1$$

$$2 < 3$$

$$D^3[3,1] = D^2[3,4] + D^2[4,1]$$

$$11 = 7 + 12$$

$$11 < 19$$

$$D^3[3,2] = D^2[3,4] + D^2[4,2]$$

$$5 = 7 + 1$$

$$5 < 8$$

### Back Tracking

- Based on a depth first recursion search.
- If test, to see whether a solution has been found and if so, return it. Otherwise make a choice and return if the recursion return a solution.
- If no chance it return is false.
- It can be applied in decision problem used to find a feasible solution of the problem.
- Optimization problems to find the best solution that can be applied.
- In backtracking problem the algorithm tries to find a sequence path to the solution.

which has some small check points for which the problem can backtrack.

### 'N' Queen's problem

\* N Queens problem is placing 'N' queens on an  $N \times N$  chess board so that no two queens attack each other.

\* We have to check that no two queens can be placed in same row, same column and same diagonal.

### Algorithm

(1): Check the row, column and same diagonal before placing the queen. If the place is safe then increment column by one, and also reinitialize the row to zero. Else

increment the row by one.

(2): If the row or column reach its maximum and the position is not safe then backtrack and remove

the last inserted queen.

(2i): Take the row, column of the last deleted queen through backtracking, increment row by one, and keep the column as same. Then check the new position is safe or not. If safe place the queen then increment the column.

Else

Increment the row.

29/06/22

Q: Place 4 Queens in a  $4 \times 4$  chess board, place the first queen in (0,0) location.

Ans:  $\rightarrow$

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | Q | X | X |   |
| 1 |   | X | X |   |
| 2 |   |   | Q | X |
| 3 |   |   | X |   |

B

4)  $r=0$   $c=2$   $B[0][2] = \text{not safe}$

5)  $r=1$   $c=2$   $B[1][2] = \text{not safe}$

6)  $r=2$   $c=2$   $B[2][2] = \text{not safe}$

7)  $r=3$   $c=2$   $B[3][2] = \text{not safe}$

row reach its maximum then backtrack & remove last inserted queen and obtain its position. deleted queen position  $(2,2)$

new position  $B[0][1]$

$r=0$   $c=1$   $B[0][1] = \text{not safe}$

2) new pos.

$r=1$   $c=1$   $B[1][1] = \text{not safe}$

3)  $r=2$   $c=1$   $B[2][1] = \text{safe}$

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | Q |   | X | X |
| 1 |   |   | Q | X |
| 2 |   |   |   | X |
| 3 |   | Q |   | X |

deleted position (2)[1]  
then increment row by 1

8)  $r=3$   $c=1$   $B[3][1] = \text{safe}$

9)  $r=0$   $c=2$   
 $B[0][2] = \text{not safe}$

(15) back track and  
remove queen at (1)[2]  
reposition  $r=0$   $c=2$

10)  $r=1$   $c=2$   
 $B[1][2] = \text{safe}$

11)  $r=0$   $c=3$   
 $B[0][3] = \text{not safe}$

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | Q |   |   |   |
| 1 |   |   |   |   |
| 2 |   |   | X |   |
| 3 |   | Q | X |   |

$B[2][2] = \text{not safe}$

12)  $r=1$   $c=3$   
 $B[1][3] = \text{not safe}$

13)  $r=2$   $c=3$   
 $B[2][3] = \text{not safe}$

14)  $r=3$   $c=3$   
 $B[3][3] = \text{not safe}$

16)  $r=3$   $c=2$   $B[3][2] = \text{not safe}$   
17) back track and remove last  
queen inserted at position  
[3][1]

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | Q |   |   |   |
| 1 |   |   |   |   |
| 2 |   |   |   |   |
| 3 |   |   |   |   |

$B[3][1]$  Not possible to increment the row  
then

(18) backtrack & remove last inserted queen  
at the position [0][0]

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 |   | X | Q | X |
| 1 | Q | X |   | X |
| 2 |   | X |   | Q |
| 3 |   | Q |   |   |

19) new pos  $r=1$   $c=0$   
 $B[1][0] = \text{safe}$

20)  $r=0$   $c=1$   $B[0][1] = \text{not safe}$

21)  $r=1$   $c=1$   $B[1][1] = \text{not safe}$

22)  $r=2$   $c=1$   $B[2][1] = \text{not safe}$

23)  $r=3$   $c=1$   
 $B[3][1] = \text{safe}$

24)  $r=0$   $c=2$   $B[0][2] = \text{not safe}$   
25)  $r=1$   $c=2$   $B[1][2] = \text{not safe}$   
26)  $r=2$   $c=2$   $B[2][2] = \text{not safe}$   
27)  $r=3$   $c=2$   $B[3][2] = \text{not safe}$

28)  $r=0$   $c=2$   
 $B[0][2] = \text{safe}$

29)  $r=1$   $c=3$   
 $B[1][3] = \text{safe}$

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | Q | Q | 1 | Q |
| 1 | 1 | Q | Q | Q |
| 2 | Q | Q | Q | 1 |
| 3 | Q | 1 | Q | Q |

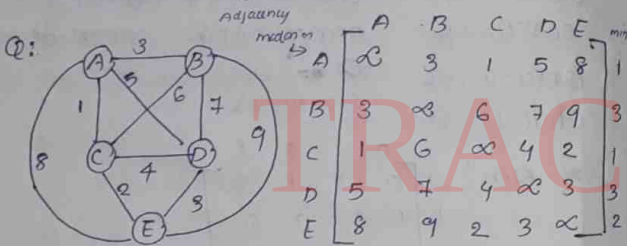
01/07/20

## Travelling Salesman Problem

using Branch and Bound.

\* The problem states that salesman should start his journey from his home city to all other cities only once and come back to his home city.

\* We have to find the minimum cost of travel taken by the salesman.



Step (1): Generate the adjacency matrix from the given graph and check whether the row and column of matrix are reduced or not. If the row or column contains 0, then we can say that column or row is reduced.

If we can't find a zero (0) then take the smallest element from each row and subtract that minimum value from the corresponding row values call row reduction.

Then the column can not reduced find the minimum value and subtract the value from column element.

To find the total cost for the reduction is sum of minimum values of row and minimum values of column.

Mark the total cost to the home city.

Row reduction

|          |          |          |          |          |   |
|----------|----------|----------|----------|----------|---|
| $\infty$ | 2        | 0        | 4        | 7        | 1 |
| 0        | $\infty$ | 3        | 4        | 6        | 3 |
| 0        | 5        | $\infty$ | 3        | 1        | 1 |
| 2        | 4        | 1        | $\infty$ | 0        | 3 |
| 6        | 7        | 0        | 1        | $\infty$ | 2 |

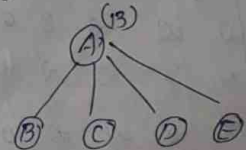
2 1  $\rightarrow$  3

Column reduction

|   | A        | B        | C        | D        | E        |
|---|----------|----------|----------|----------|----------|
| A | $\infty$ | 0        | 0        | 3        | 7        |
| B | 0        | $\infty$ | 3        | 3        | 6        |
| C | 0        | 3        | $\infty$ | 2        | 1        |
| D | 2        | 2        | 1        | $\infty$ | 0        |
| E | 6        | 5        | 0        | 0        | $\infty$ |

Input matrix

Total cost of city A =  $7 + 0 + 3 = 13$



Step (2): A to B, A → row, B → column.  
make A row ∞ and B row ∞ path  
from B to A row ∞

| A | B | C | D | E |
|---|---|---|---|---|
| ∞ | ∞ | ∞ | ∞ | ∞ |
| ∞ | ∞ | 3 | 3 | 6 |
| 0 | ∞ | ∞ | 2 | 1 |
| 2 | ∞ | 1 | ∞ | 0 |
| 6 | ∞ | 0 | 0 | ∞ |

column reduce

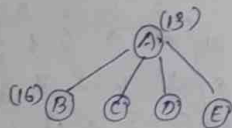
claim (A, B)

$$\text{reduction of } (A, B) = (3+0) = 3$$

Total cost of travel from A to B =  $13+9+0$

$$= 13+9+0$$

$$= 16$$



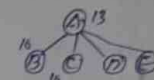
Step (3): A to C

| A | B | C | D | E |
|---|---|---|---|---|
| ∞ | ∞ | ∞ | ∞ | ∞ |
| 0 | ∞ | ∞ | 3 | 6 |
| ∞ | 3 | ∞ | 2 | 1 |
| 2 | 2 | ∞ | ∞ | 0 |
| 6 | 5 | ∞ | 0 | ∞ |

| A | B | C | D | E |
|---|---|---|---|---|
| ∞ | ∞ | ∞ | ∞ | ∞ |
| 0 | ∞ | 3 | 3 | 6 |
| ∞ | 0 | ∞ | 1 | 0 |
| 2 | 0 | ∞ | ∞ | 0 |
| 6 | 3 | ∞ | 0 | ∞ |

Total reduction cost (A, D)  
=  $1+2 = 3$

Total cost from A to C =  $13+9+2$   
=  $13+3+0$   
=  $16$



Step (4): A to D

| A | B | C | D | E |
|---|---|---|---|---|
| ∞ | ∞ | ∞ | ∞ | ∞ |
| 0 | ∞ | 3 | ∞ | 6 |
| ∞ | 0 | ∞ | ∞ | 1 |
| ∞ | 2 | 1 | ∞ | 0 |
| 6 | 5 | 0 | ∞ | ∞ |

| A | B | C | D | E |
|---|---|---|---|---|
| ∞ | ∞ | ∞ | ∞ | ∞ |
| 0 | ∞ | 3 | ∞ | 6 |
| 0 | 1 | ∞ | ∞ | 1 |
| ∞ | 0 | 1 | ∞ | 0 |
| 6 | 3 | 0 | ∞ | ∞ |

reduction cost (A, D) =  $0+2 = 2$

Total cost from A to D =  $13+9+2$

$$= 13+2+3 = 18$$

Step (5): A to E

| A | B | C | D | E |
|---|---|---|---|---|
| ∞ | ∞ | ∞ | ∞ | ∞ |
| 0 | ∞ | 3 | 3 | ∞ |
| ∞ | 0 | 3 | 2 | ∞ |
| 2 | 2 | 1 | ∞ | ∞ |
| ∞ | 5 | 0 | 0 | ∞ |

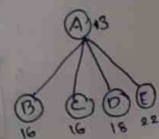
| A | B | C | D | E |
|---|---|---|---|---|
| ∞ | ∞ | ∞ | ∞ | ∞ |
| 0 | ∞ | 3 | 3 | ∞ |
| 0 | 3 | 2 | ∞ | ∞ |
| 1 | 1 | 0 | ∞ | ∞ |
| ∞ | 5 | 0 | 0 | ∞ |

| A | B | C | D | E |
|---|---|---|---|---|
| ∞ | ∞ | ∞ | ∞ | ∞ |
| 0 | ∞ | 3 | 3 | ∞ |
| 0 | 2 | ∞ | 2 | ∞ |
| 0 | 1 | 0 | ∞ | ∞ |
| ∞ | 4 | 0 | 0 | ∞ |

reduction cost (A, E) =  $1+1 = 2$

Total cost from A to E =  $13+9+2$

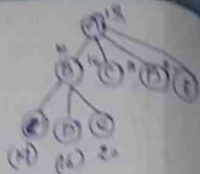
$$= 13+2+3 = 18$$



Step 1:

reduced matrix of AB

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | ∞ | ∞ | 0 | 0 | 5 |
| C | 0 | ∞ | ∞ | 2 | 1 |
| D | 2 | ∞ | 1 | ∞ | 0 |
| E | 6 | ∞ | 0 | 0 | ∞ |



AB to C B → row C → column is as follows

CA also is

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | ∞ | ∞ | ∞ | ∞ | ∞ |
| C | ∞ | ∞ | ∞ | ∞ | ∞ |
| D | 2 | ∞ | ∞ | ∞ | 0 |
| E | 6 | ∞ | ∞ | ∞ | ∞ |

row sum →

column sum →

Reduction cost (B,C) = 9 + 10 = 19

Total cost sum = 16 + 3 + 0 = 19

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | ∞ | ∞ | ∞ | ∞ | ∞ |
| C | ∞ | ∞ | ∞ | 1 | 0 |
| D | 2 | ∞ | ∞ | ∞ | 0 |
| E | 6 | ∞ | ∞ | ∞ | ∞ |

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | ∞ | ∞ | ∞ | ∞ | ∞ |
| C | 0 | ∞ | ∞ | ∞ | ∞ |
| D | ∞ | ∞ | 1 | ∞ | 0 |
| E | ∞ | ∞ | 0 | ∞ | ∞ |

row = 0

column = 0

Reduction cost → 0

Total Cost sum (Bd)

= 16 + 0 + 0 = 16

AB to E

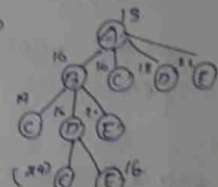
|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | ∞ | ∞ | ∞ | ∞ | ∞ |
| C | 0 | ∞ | ∞ | 2 | ∞ |
| D | 2 | ∞ | 1 | ∞ | ∞ |
| E | ∞ | ∞ | 0 | 0 | ∞ |

row = 0 column = 0 Reduction cost (B,E) = 1

Total cost sum (B,E) = 16 + 1 + 3 = 19

Step (4): reduced matrix ABD

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | ∞ | ∞ | ∞ | ∞ | ∞ |
| C | 0 | ∞ | ∞ | ∞ | ∞ |
| D | ∞ | ∞ | 1 | ∞ | 0 |
| E | 6 | ∞ | 0 | ∞ | ∞ |



D → row column. all ∞

ABD - C

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | ∞ | ∞ | ∞ | ∞ | ∞ |
| C | ∞ | ∞ | ∞ | ∞ | ∞ |
| D | ∞ | ∞ | ∞ | ∞ | ∞ |
| E | 6 | ∞ | ∞ | 0 | ∞ |

Reduction cost = 1 + 6 = 7

Total cost = 16 + 7 + 1 = 24

